Hastig-H7-1 Integration Manual

1. General Overview Of Software Architecture

1.1 Runtime model

The firmware runs as a hybrid of:

- Arduino `setup()` and `loop()`
- Multiple Mbed RTOS threads
- Mailbox/event-based communication between modules

Main entry points:

- `src/main.cpp`
- `include/SystemContext.h`

1.2 Main components

- `Orchestrator` (`src/Orchestrator.cpp`)
- Device state machine (`aware`, `sampling`, `hibernating`)
- Command handling
- Timeout logic (inactivity, no-network, unacked aggregate fallback)
- `CommsPump` (`src/CommsPump.cpp`)
- GSM + MQTT connectivity
- MQTT subscribe/publish
- Routes inbound MQTT to settings or orchestrator
- `SamplingThread` (`src/SamplingThread.cpp`)
- Creates/owns sensor instance
- Produces `SensorSampleMsg`
- `AggregatorThread` (`src/AggregatorThread.cpp`)
- Consumes samples, builds aggregate window
- Emits `AggregateMsg`
- `UiThread` + `LcdMenu` (`src/UiThread.cpp`, `src/LcdMenu.cpp`)
- Button handling
- Status screen + local setup menu
- `SettingsManager` (`src/SettingsManager.cpp`)
- Thread-safe runtime settings
- Flash persistence
- `PowerManager` (`src/PowerManager.cpp`)
- Executes controlled sleep transaction

1.3 Internal data flow

1. Sensor pipeline:
2. `SamplingThread` -> mailbox `sensorToAggMail` -> `AggregatorThread` -> mailbox `aggToCommsMail` -> `CommsPump` -> MQTT `/data`
3. Command pipeline:
4. MQTT `/cmd` -> `CommsPump` -> `EventBus` -> `Orchestrator`
5. Remote config pipeline:
6. MQTT `/cfg` -> `CommsPump` -> `SettingsManager.applyJson(..., persist=true)`
7. Local UI setup pipeline:
8. UI menu `topic=setup` -> `UiThread` -> `Orchestrator` -> `CommsEgress.applySettingsJson(...)` -> `CommsPump` -> `SettingsManager`

1.4 State machine

- `aware`
- Network connected, waiting for commands
- Periodic status reporting
- `sampling`
- Sampling + aggregation enabled
- Sends aggregate packets
- Returns to `aware` if `maxUnackedPackets` attempted publishes occur without `keepSampling`
- `hibernating`
- Publish hibernation status/mode change
- `PowerManager` performs sleep sequence

1.5 MQTT topic model

Topic format:

`<prefix>/<nodeId>/<postfix>`

Defaults from `include/AppConfig.h`:

- `prefix`: `hastigNode`
- command postfix: `cmd`
- config postfix: `cfg`
- data postfix: `data`
- status postfix: `status`

`nodeId` source:

- `settings.device_name` if non-empty
- otherwise hardware ID (`BoardHal::getHardwareId` , 24 hex chars)

2. MQTT Integration Contract

2.1 Common rules

- Payloads are JSON objects.
- Unknown keys are ignored.
- Most numeric fields are integer unless explicitly float.
- Device subscribes to:
- `<prefix>/<nodeId>/cmd`
- `<prefix>/<nodeId>/cfg`
- Device publishes to:
- `<prefix>/<nodeId>/status`
- `<prefix>/<nodeId>/data`

2.2 Incoming messages (`/cmd`)

Mandatory for all command messages:

- `type` (string)

Supported `type` values and optional keys:

1. `keepSampling`
2. Optional keys: none
3. `startSampling`
4. Optional keys:
    - `samplingInterval` (uint, ms)
    - `aggPeriodS` (uint, seconds)
    - `sessionID` (string)
5. `stopSampling`
6. Optional keys: none
7. `getConfig`
8. Optional keys: none
9. `hibernate`
10. Optional keys:
    - `sleepSeconds` (uint)
11. `nudge`
12. Optional keys: none
13. `resetBatteryStatistics`

14. Optional keys: none
15. `factoryReset`
16. Optional keys: none

Example:

```
{"type":"startSampling","samplingInterval":1000,"aggPeriodS":30,"ses
```

2.3 Incoming messages (`/cfg`)

`/cfg` payload is a JSON patch. No key is mandatory; include only keys to update.

Recognized keys:

- `sensorAddress` (uint8)
- `sensorBaudrate` (uint32)
- `sensorWarmupMs` (uint32)
- `sensorType` (uint32)
- `samplingInterval` (uint32, ms)
- `aggPeriodS` (uint32, s)
- `aggregationMethod` (string; currently accepted but not functionally used)
- `simPin` (string)
- `apn` (string)
- `apnUser` (string)
- `apnPass` (string)
- `mqttHost` (string)
- `mqttPort` (uint16)
- `mqttUser` (string)
- `mqttPass` (string)
- `mqttClientId` (string)
- `deviceName` (string)
- `awareTimeoutS` (uint32)
- `defaultSleepS` (uint32)
- `statusIntervalS` (uint32)
- `lowBattMinV` (float)
- `maxChargingCurrent` (uint16)
- `maxChargingVoltage` (float)
- `emergencyDelayS` (uint32)
- `emergencySleepS` (uint32)
- `maxForcedSleepS` (uint32)
- `maxUnackedPackets` (uint32)

Example:

```
{"samplingInterval":500,"aggPeriodS":20,"sensorAddress":3}
```

2.4 Outgoing messages (`/status`)

`/status` is multiplexed and carries several message types via `type`.

A) `type = "status"`

Mandatory keys:

- `type` = `"status"`
- `tsMs` (uint32)
- `mode` (`aware` | `sampling` | `hibernating`)

Optional keys (present for periodic battery status):

- `batteryVoltage` (float)
- `minimumVoltage` (float)
- `batteryCurrent` (float)
- `averageCurrent` (float)

Optional keys (present for hibernate status):

- `reason` (string)
- `expectedDuration` (uint32, seconds)

Example:

```
{"type":"status","tsMs":123456,"mode":"sampling","batteryVoltage":3.
```

B) `type = "modeChange"`

Mandatory keys:

- `type` = `"modeChange"`
- `previousMode` (string)

Optional keys:

- `mode` (string)
- `reason` (string; used for hibernation transitions)

- `expectedDuration` (uint32; used for hibernation transitions)

Example:

```
{"type":"modeChange","mode":"hibernating","previousMode":"aware","re
```

C) `type = "alert"`

Mandatory keys:

- `type` = `"alert"`
- `message` (string)
- `mode` (string)
- `minimumVoltage` (float)

Example:

```
{"type":"alert","message":"Critically low battery detected. Emergenc
```

D) `type = "config"`

Mandatory keys:

- `type` = `"config"`
- `tsMs` (uint32)

Optional keys:

- Any masked config fields (single-message snapshot when payload is small enough)

E) `type = "configChunk"`

Mandatory keys:

- `type` = `"configChunk"`
- `tsMs` (uint32)
- `chunk` (uint8, 1-based)
- `total` (uint8)
- `section` (`network` | `mqtt` | `device` | `schedule` | `power`)

Optional keys:

- Masked config keys for that section

Notes:

- Secrets are masked as `"***"` when set (`simPin` , `apnUser` , `apnPass` , `mqttUser` , `mqttPass`).

2.5 Outgoing messages (`/data`)

`/data` carries aggregated sensor windows.

Mandatory keys:

- `type` = `"data"`
- `t0` (uint32, relative start ms)
- `t1` (uint32, relative end ms)
- `n` (uint32, sample count in window)
- `ok` (0 | 1)
- `<k0>Avg` (float)
- `<k0>Min` (float)
- `<k0>Max` (float)

Optional keys:

- `<k1>Avg` (float)
- `<k1>Min` (float)
- `<k1>Max` (float)
- present when second metric exists

Typical metric keys from current sensors:

- `k0 = "cond"`
- `k1 = "temp"`

Example:

```
{"type":"data","t0":10000,"t1":25000,"n":15,"ok":1,"condAvg":1.94,"c
```

3. Local Display Menu Structure

Source of truth: `include/MenuDef.h` .

3.1 Display modes and key behavior

There are two display modes:

1. `status` mode (default at boot)
2. `menu` mode

Button behavior:

- In `status` mode:
- `Left` or `Right` enters menu mode
- In `menu` mode:
- `Up` / `Down` : move selection
- `Right` : confirm/select
- `Left` : back
- `Left` at root: return to status mode

3.2 Top-level menu tree

1. `Start sampling` (action -> `cmd:startSampling`)
2. `Stop sampling` (action -> `cmd:stopSampling`)
3. `Hibernate 10m` (action -> `cmd:hibernate`)
4. `Setup` (submenu)
5. `Send config` (action -> `cmd:getConfig`)

3.3 Setup submenu

Numeric/select items:

1. `Sensor address` -> `sensorAddress` options: `1..9`
2. `Sensor baudrate` -> `sensorBaudrate` options:
3. `1200, 4800, 9600, 19200, 38400, 57600, 115200`
4. `Sensor warmup (ms)` -> `sensorWarmupMs` options:
5. `500, 1000, 2000, 4000, 5000, 10000`
6. `Sensor type` -> `sensorType` options:
7. `0, 1, 2`
8. `Sampling interval` -> `samplingInterval` options:
9. `250, 500, 750, 1000, 2000, 5000, 10000`
10. `Agg period (s)` -> `aggPeriodS` options:
11. `10, 15, 20, 30, 60, 120, 300, 600`
12. `Aware timeout (s)` -> `awareTimeoutS` options:
13. `60, 300, 600, 900, 1800, 3600`
14. `Default sleep (s)` -> `defaultSleepS` options:
15. `600, 900, 1800, 3600`
16. `Status interval (s)` -> `statusIntervalS` options:
17. `60, 120, 300, 600, 900, 1800`
18. `Max charge current` -> `maxChargingCurrent` options:
    - `250, 500, 750, 1000`

19. `Max charge volt` -> `maxChargingVoltage` options:
    - `3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2`

Text editor items:

1. `Device name` -> `deviceName` (maxLen 47)
2. `SIM pin` -> `simPin` (maxLen 15)
3. `APN` -> `apn` (maxLen 63)
4. `APN user` -> `apnUser` (maxLen 31)
5. `APN pass` -> `apnPass` (maxLen 31)
6. `MQTT host` -> `mqttHost` (maxLen 63)
7. `MQTT user` -> `mqttUser` (maxLen 31)
8. `MQTT pass` -> `mqttPass` (maxLen 31)
9. `MQTT client id` -> `mqttClientId` (maxLen 47)

3.4 Text editor behavior (for text settings)

- Character grid includes:
- letters, digits, `.`, `-`, `_`, and space
- Command cells:
- `<` delete
- `^` toggle case
- `+` confirm/save
- `x` cancel
- Trailing spaces are trimmed on save.
- Saved text updates local settings; it is not automatically pushed to backend.

Note:

- The menu item label `Hibernate 10m` sends `cmd:hibernate` without `sleepSeconds`.
- Actual sleep duration then follows runtime setting `defaultSleepS` (unless command payload provides `sleepSeconds` from backend).